

Local Extreme Learning Machines and Domain Decomposition for Solving Linear and Nonlinear Partial Differential Equations

Summary of Dong and Li (2021)

1 Introduction

Neural network-based methods for solving partial differential equations (PDEs) have attracted significant research interest in recent years. However, existing deep neural network (DNN) methods like Deep Galerkin Method (DGM) and Physics-Informed Neural Networks (PINN) suffer from limitations in accuracy and computational efficiency. This paper introduces a method called Local Extreme Learning Machines (locELM) that combines domain decomposition with shallow neural networks to achieve superior performance for solving linear and nonlinear PDEs.

The locELM method addresses key limitations of existing DNN-based PDE solvers:

- It achieves exponential or near-exponential convergence with respect to degrees of freedom
- It produces highly accurate solutions (orders of magnitude better than DGM/PINN)
- It requires significantly less computational time than traditional DNN approaches
- It performs comparably to or better than classical finite element methods (FEM)

2 Algorithm Description

2.1 Domain Decomposition and Local Network Architecture

The locELM method represents a novel approach to solving PDEs by combining three key ideas:

1. Domain decomposition

2. Local neural networks
3. Extreme learning machine (ELM) training

Given a domain Ω in d dimensions (where one dimension may represent time), the method:

1. Partitions Ω into N_e non-overlapping sub-domains:

$$\Omega = \Omega_1 \cup \Omega_2 \cup \cdots \cup \Omega_{N_e} \quad (1)$$

2. Assigns a separate shallow feed-forward neural network to each sub-domain Ω_i to represent the local solution $f_i(x)$
3. Imposes C^k continuity conditions across sub-domain boundaries (where k depends on the order of the PDE)
4. Uses shallow neural networks with a small number of hidden layers (typically 1-3), but allows the last hidden layer to be wide

Each local neural network has specific characteristics:

- The weight/bias coefficients for all hidden layers are pre-set to random values in $[-R_m, R_m]$ and remain fixed
- Only the weights of the output layer are trainable parameters
- The output layer is linear (no activation function or bias)

The local representation on sub-domain Ω_s is:

$$u_i^s(x) = \sum_{j=1}^M V_j^s(x) w_{ji}^s, \quad x \in \Omega_s, \quad 1 \leq i \leq N \quad (2)$$

where $V_j^s(x)$ is the output of the last hidden layer, w_{ji}^s are the trainable parameters, and M is the number of nodes in the last hidden layer.

2.2 Training Process for Linear PDEs

For linear PDEs, the training process involves:

1. Placing collocation points within each sub-domain and on boundaries
2. Enforcing the PDE on interior collocation points:

$$\sum_{j=1}^M [LV_j^{emn}(x_p^{emn}, y_q^{emn})] w_j^{emn} = f(x_p^{emn}, y_q^{emn}) \quad (3)$$

3. Enforcing boundary conditions on domain boundaries
4. Enforcing C^k continuity conditions across sub-domain boundaries
5. Formulating these constraints as a system of linear equations
6. Solving the resulting system using linear least squares minimization

The continuity conditions are crucial for coupling the local neural networks together, allowing information to propagate between sub-domains while maintaining the required smoothness.

2.3 Block Time-Marching for Time-Dependent PDEs

For long-time simulations of time-dependent PDEs, the method employs a block time-marching strategy:

1. The spatial-temporal domain is divided into windows or "time blocks" along the temporal direction
2. The PDE is solved on each time block separately and successively
3. The solution from one time block at the final time instant serves as the initial condition for the next time block
4. Each time block uses its own set of local neural networks

This strategy significantly improves training for long-time simulations, especially for nonlinear PDEs, where solving over the entire temporal domain at once would be challenging or impossible.

2.4 Algorithm for Nonlinear PDEs

For nonlinear PDEs, the process is similar, but two methods are proposed for solving the resulting nonlinear system:

1. **NLSQ-perturb** (Nonlinear Least Squares with Perturbations):
 - Uses nonlinear least squares optimization
 - Incorporates random perturbations to avoid local minima when needed
 - Generally produces more accurate results but is computationally more expensive
2. **Newton-LLSQ** (Newton-Linear Least Squares):
 - Linearizes the PDE using Newton's method

- Solves for the increment field using linear least squares
- Computationally faster but typically less accurate than NLSQ-perturb

The NLSQ-perturb method is particularly important for preventing the algorithm from being trapped in local minima, especially in under-resolved cases and long-time simulations.

3 Theoretical Findings

3.1 Convergence Properties

The locELM method exhibits several notable convergence properties:

- **Exponential convergence:** Numerical errors typically decrease exponentially or nearly exponentially as:
 - The number of sub-domains increases
 - The number of collocation points per sub-domain increases
 - The number of training parameters per sub-domain increases
- **Effect of random coefficients:** The accuracy is influenced by R_m , the maximum magnitude of random coefficients
 - Very large or very small values of R_m adversely affect accuracy
 - Optimal accuracy is achieved with a moderate range of R_m values (typically around $R_m \approx 1 - 10$)
 - The optimal range expands and shifts as simulation resolution increases

3.2 Domain Decomposition Benefits

The use of domain decomposition with local neural networks provides significant benefits:

- **Reduced computational cost:** With the same total degrees of freedom, increasing the number of sub-domains significantly reduces training time
- **Sparse coefficient matrix:** The locELM approach creates a sparse coefficient matrix in the least squares problem, as only neighboring sub-domains are coupled
- **Maintained accuracy:** The accuracy with multiple sub-domains is comparable to (and sometimes better than) using a single global domain with the same total degrees of freedom

3.3 Multiple Hidden Layers

Unlike traditional ELM methods that are limited to a single hidden layer, the locELM approach allows multiple hidden layers in each local neural network:

- Local networks with 1-3 hidden layers have been successfully tested
- Using more than one hidden layer can produce accurate results with appropriate settings
- Increasing beyond three hidden layers tends to make accurate results harder to achieve

4 Numerical Examples and Results

The authors conducted extensive numerical experiments with various PDEs to demonstrate the performance of the locELM method.

4.1 Linear PDEs

4.1.1 Helmholtz Equation (1D)

$$\frac{d^2u}{dx^2} - \lambda u = f(x) \quad (4)$$

Key findings:

- Error decreased from $\sim 10^1$ with a single sub-domain to $\sim 10^{-7}$ with 8 sub-domains
- Exponential decrease in errors with increasing collocation points and training parameters
- Quadrature-point distribution of collocation points produced more accurate results than uniform distribution, which in turn was more accurate than random distribution

4.1.2 Advection Equation (1D + time)

$$\frac{\partial u}{\partial t} - c \frac{\partial u}{\partial x} = 0 \quad (5)$$

Key findings:

- Successfully performed long-time simulations (up to 100 time units, approximately 40 periods)
- Block time-marching was crucial for long-time accuracy
- Training time with 4 sub-domains per time block was less than half that of a single sub-domain

4.1.3 Diffusion Equation (1D/2D + time)

$$\frac{\partial u}{\partial t} - \nu \frac{\partial^2 u}{\partial x^2} = f(x, t) \quad (6)$$

Key findings:

- Errors decreased from $\sim 10^{-1}$ with one sub-domain to $\sim 10^{-8}$ with 5 sub-domains
- Successfully extended to 2D spatial problems plus time
- Achieved error levels of $\sim 10^{-6}$ for the 2D+time case

4.2 Nonlinear PDEs

4.2.1 Nonlinear Helmholtz Equation (1D)

$$\frac{d^2 u}{dx^2} - \lambda u + \beta \sin(u) = f(x) \quad (7)$$

Key findings:

- NLSQ-perturb achieved error levels of $\sim 10^{-12}$ to 10^{-9}
- Newton-LLSQ achieved error levels of $\sim 10^{-9}$ to 10^{-5}
- Newton-LLSQ was generally faster but less accurate than NLSQ-perturb

4.2.2 Nonlinear Spring Equation (time only)

$$\frac{d^2 u}{dt^2} + \omega^2 u + \alpha \sin(u) = f(t) \quad (8)$$

Key findings:

- Successfully performed long-time simulations (up to 100 time units)
- Achieved error levels of $\sim 10^{-8}$ with block time-marching
- Exponential decrease in errors with increasing collocation points (until saturation)

4.2.3 Viscous Burger's Equation (1D + time)

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} + f(x, t) \quad (9)$$

Key findings:

- Achieved error levels of $\sim 10^{-8}$ to 10^{-7} with NLSQ-perturb
- Block time-marching essential for accurate long-time simulations
- Error decreased approximately exponentially with increasing collocation points

4.3 Comparative Performance Analysis

4.3.1 Comparison with DGM and PINN

The locELM method demonstrated clear superiority over DGM and PINN:

- **Accuracy:**

- For the 1D Helmholtz equation: locELM errors ($\sim 10^{-9}$) were about 5 orders of magnitude smaller than DGM/PINN errors ($\sim 10^{-4}$)
- For the 1D diffusion equation: locELM errors ($\sim 10^{-8}$) were about 5 orders of magnitude smaller than DGM/PINN errors ($\sim 10^{-3}$)
- For the nonlinear spring: locELM errors ($\sim 10^{-11}$) were about 6 orders of magnitude smaller than PINN errors ($\sim 10^{-5}$)
- For the Burger’s equation: locELM errors ($\sim 10^{-8}$) were about 5 orders of magnitude smaller than DGM/PINN errors ($\sim 10^{-3}$)

- **Computational cost:**

- For the 1D Helmholtz equation: locELM training time (1.1 seconds) was about 1000 times faster than PINN training time (1035.8 seconds)
- For the 1D diffusion equation: locELM training time (28.4 seconds) was about 100 times faster than PINN training time (3174.4 seconds)
- For Burger’s equation: locELM training time (27.6 seconds) was about 65 times faster than DGM training time (1813.5 seconds)

4.3.2 Comparison with Finite Element Method (FEM)

The locELM method was competitive with or superior to classical FEM:

- **Similar accuracy-cost performance:**

- For the 1D Helmholtz equation: locELM with 4 sub-domains and 100 collocation points achieved higher accuracy than FEM with 100,000 elements in comparable computation time
- For the 1D diffusion equation: locELM with 5 sub-domains achieved about 3 orders of magnitude better accuracy than FEM with similar computational cost
- For Burger’s equation: locELM with NLSQ-perturb achieved 3 orders of magnitude better accuracy than FEM with comparable computational cost

- **Qualitatively different convergence behavior:**
 - FEM showed algebraic convergence (second-order for linear elements)
 - locELM exhibited exponential convergence until reaching a saturation point
 - For higher-order FEM (degree 3-4 Lagrange elements), locELM remained competitive but not consistently superior

5 Implications and Advantages

The locELM method offers several significant advantages:

1. **Superior accuracy:** Achieves exponential convergence and significantly higher accuracy than DNN-based PDE solvers
2. **Computational efficiency:** Requires orders of magnitude less training time than traditional DNN-based methods and is competitive with classical numerical methods
3. **Enhanced trainability:** The domain decomposition approach leads to sparser coefficient matrices and better trainability, especially for complex problems
4. **Long-time simulation capability:** The block time-marching strategy enables accurate long-time simulations of time-dependent PDEs
5. **Versatility:** Successfully handles both linear and nonlinear, stationary and time-dependent PDEs in multiple dimensions

The method provides evidence that neural network-based approaches can be competitive with or even outperform traditional numerical methods for solving PDEs. The superior performance of locELM compared to DGM and PINN suggests that:

- Pre-setting random weights in hidden layers and only training output layer weights can be more effective than training all network parameters
- Domain decomposition with local neural networks offers significant advantages over global networks
- Linear/nonlinear least squares training is computationally more efficient than gradient descent-based training for PDE problems

The locELM approach represents a significant step toward making neural network-based methods truly competitive in computational science and engineering applications, potentially opening new pathways for solving complex PDEs efficiently and accurately.